Since 1983

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Department of Computer Engineering and Technology

MINI PROJECT REPORT

Computer Networks Laboratory (CSE20110)

Laboratory Continuous Assessment 3

NetPulse: Intelligent Network Traffic Management, Adaptive Load Balancing & Fault-Tolerant Monitoring System

Submitted by

Sr. No.	Student Name	PRN
1	Naitik Sharma	1262243087
2	Purva Shetiya	1262243040
3	Tanushree Walunj	1262243564
4	Shubham Ojha	1262241817

Group No. Panel J Batch J1

Third Year B.Tech. Computer Engineering and Technology

Under the guidance of

Dr. Uma Pujeri

Academic Year 2026–2027

Submission Date [21/10/2026]

Since 1983

Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

MIT-WPU
॥ विश्वशान्तिर्युवा ॥

Department of Computer Engineering and Technology

CERTIFICATE

This is to certify that the mini-project entitled "NetPulse: Intelligent Network Traffic Management, Adaptive Load Balancing & Fault-Tolerant Monitoring System" has been carried out by the following students of Third Year B.Tech. Computer Science and Engineering, Panel [PANEL], Batch [BATCH], Group No. [GROUP NUMBER], under my guidance during the academic year 2026–2027.

Student Name	**PRN**
Naitik Sharma	1262243087
Purva Shetiya	1262243040
Tanushree Walunj	1262243564
Shubham Ojha	1262241817

The report is submitted towards the mini-project component of Laboratory Continuous Assessment 3 for the Computer Networks Laboratory course.

Faculty Signature _____

Dr. Uma Pujeri
Project Guide and Course Faculty

Date _____ Place: Pune

Acknowledgement

The team expresses its sincere gratitude to the Department of Computer Engineering and Technology, MIT World Peace University, Pune, for providing the academic environment and laboratory context for this Computer Networks mini-project. We thank our project guide and course faculty for their guidance and feedback. The project uses open-source software and documentation from the Python, FastAPI, HTTPX, pytest and Wireshark ecosystems. The team is responsible for verifying the final implementation, results, references and individual contribution details.

Group No.

Abstract

NetPulse is an academic implementation of an asynchronous Layer-7 HTTP reverse proxy, dynamic load balancer and fault-monitoring system developed for the Computer Networks Laboratory (CSE20110). It receives HTTP requests on port 8080 and distributes them across three local backend servers on ports 5001, 5002 and 5003. Four scheduling strategies are available: Round Robin, Smooth Weighted Round Robin, weighted Least Connections and IP Hash. An asynchronous health monitor probes backend health endpoints, while passive proxy failures can contribute to failure detection. The system also provides an Admin REST API on port 9090, configurable timeouts and retries, request tracing headers, bounded latency samples, latency percentiles, a monitoring dashboard and a Demo Store that generates real proxy traffic. A separate TCP stream proxy demonstrates Layer-4 forwarding. The uploaded project snapshot passed 44 automated pytest tests in the review environment. A saved benchmark run containing 90 requests at concurrency 9 per algorithm reported zero failed requests for Round Robin, Smooth Weighted Round Robin and Least Connections; observed throughput and latency varied by strategy. These measurements describe one local test run, not a universal performance guarantee. NetPulse demonstrates practical networking concepts while retaining limitations including local in-memory control state, single-proxy availability, simulated checkout, and the need for further concurrency and recovery testing.

Keywords: Reverse Proxy, Load Balancing, FastAPI, Health Monitoring, TCP, HTTP, Fault Tolerance, Observability

Table of Contents

Open in Microsoft Word, right-click this table and choose Update Field → Update entire table.

List of Figures

- Figure 2.1 NetPulse system architecture
- Figure 3.1 Health state behavior as implemented
- Figure 4.1 Saved algorithm comparison plot

List of Tables

- Table 2.1 Tools and system requirements
- Table 2.2 Addressing and protocol selection
- Table 3.1 Core module responsibilities
- Table 4.1 Test plan and observed outcomes
- Table 4.2 Saved benchmark results
- Table 4.3 Objective achievement
- Table 5.1 Troubleshooting summary
- Table C.1 Suggested team contribution map

1 Introduction and Project Objectives

1.1 Introduction and Background

Modern web applications commonly use multiple application servers to distribute incoming client requests, improve resource utilization and reduce dependence on one server. A reverse proxy provides a single client-facing endpoint and forwards requests to upstream servers. At Layer 7, an HTTP proxy can inspect request methods, paths and headers; at Layer 4, a TCP proxy forwards byte streams without interpreting HTTP semantics [1].

NetPulse implements these ideas in Python using FastAPI, Uvicorn, asyncio and HTTPX. Its client-facing HTTP service listens on 127.0.0.1:8080. A separate administrative service listens on 127.0.0.1:9090. Three configurable backend servers listen on ports 5001, 5002 and 5003. The project includes load-balancing strategies, health probes, request metrics, a dashboard, a demo web application, test utilities and a TCP stream proxy.

HTTP forwarding also requires care with connection-specific headers. HTTP semantics and intermediary behavior are described in RFC 9110 [1]. Asynchronous I/O is useful for I/O-bound proxy work because a task can yield while awaiting network activity instead of blocking a thread for every outstanding request [2], [3].

1.2 Problem Statement

When requests are sent directly to a single application server, that server can become overloaded or unavailable. A simple round-robin dispatcher improves distribution but does not account for unequal backend capacity, active work, or server health. A laboratory implementation is needed to demonstrate how traffic scheduling, health detection, failover, request tracing and performance measurement work together.

NetPulse addresses this problem by placing a configurable reverse proxy between clients and a small backend fleet. The project makes request routing observable and provides testable control APIs and failure workflows.

1.3 Scope

The project is designed for local academic demonstration on a developer machine. It uses loopback addresses and separate TCP ports rather than a production multi-host deployment. The Demo Store is an educational storefront for simulated orders; it does not process real payments. The primary L7 path is HTTP, and the optional L4 component forwards raw TCP streams.

Scope includes four scheduling algorithms, active health probes, passive failure signals, administrative backend management, request/latency metrics, CSV and chart generation, unit/integration tests, Wireshark guidance, and optional Mininet-based network impairment. The project does not claim production-grade high availability, TLS termination, distributed service discovery, or a replicated metrics store.

1.4 Objectives

ID	Objective	Verification method

O1	Distribute HTTP requests across three backends using four scheduling strategies.	Algorithm unit tests and runtime requests.
O2	Detect backend failures and exclude unhealthy nodes from normal selection.	Health and failover tests; controlled backend stop.
O3	Expose runtime backend and algorithm controls through an Admin REST API.	Integration tests for admin endpoints.
O4	Measure request volume, errors, latency percentiles and backend distribution.	Metrics tests, benchmark CSV and generated plot.
O5	Provide a browser dashboard and Demo Store that send traffic through the proxy.	Route and Demo Store tests; browser verification.
O6	Demonstrate transport-layer stream forwarding and packet inspection.	TCP proxy test and Wireshark capture workflow.

2 Proposed Design and Requirements

2.1 Architecture and Network Topology

Figure 2.1 shows the local client-to-proxy path, administrative API, backend pool, health monitor and metrics outputs. The browser and load generator send requests to port 8080. The proxy selects an eligible backend and forwards the request using HTTPX. The Admin API on port 9090 updates backend configuration and returns metrics. The health monitor sends periodic GET /health probes to the backend ports. The dashboard consumes admin data and visualizes current status.

2.2 Tools and System Requirements

Tool / resource	Version or specification	Purpose
Python	3.10+ stated by project; tested in review environment	Application and test runtime.
FastAPI	requirements: >=0.115,<1	ASGI applications and REST endpoints.
Uvicorn	uvicorn[standard] >=0.30,<1	Runs the proxy and Admin API.
HTTPX	>=0.27,<1	Async upstream requests and test clients.
PyYAML	>=6.0,<7	Reads config.yaml.
pytest / pytest-asyncio	pytest >=8; asyncio plugin >=0.24	Automated unit and integration tests.
Matplotlib	>=3.9,<4	Benchmark plots.
HTML/CSS/JavaScript	Browser-native	Demo Store and dashboard.
Chart.js	Referenced by dashboard	Client-side chart rendering.
Wireshark / Npcap	Optional local capture setup	Inspect loopback TCP/HTTP traffic.
Mininet	Optional Linux environment	Topology and delay/loss experiments.

2.3 Addressing and Protocol Selection

Component	IP / interface	Port	Role
Client / browser	127.0.0.1	ephemeral → 8080	Sends HTTP requests.
L7 proxy	127.0.0.1	8080/TCP	HTTP reverse proxy and public app routes.

Admin API	127.0.0.1	9090/TCP	Backend control, algorithm switching, health and metrics.
Backend b1	127.0.0.1	5001/TCP	Upstream service; configured weight 5, 50 ms artificial delay.
Backend b2	127.0.0.1	5002/TCP	Upstream service; configured weight 3, 100 ms artificial delay.
Backend b3	127.0.0.1	5003/TCP	Upstream service; configured weight 1, 200 ms artificial delay.
L4 TCP proxy	127.0.0.1	8081/TCP by default	Optional raw TCP stream forwarding.

2.4 Protocol and Design Decisions

HTTP is used for the main proxy because the scheduler can route requests while preserving application-level paths, methods and headers. HTTPX provides asynchronous upstream request handling and connection pooling [3]. TCP underlies each HTTP connection; a client-to-proxy connection and a proxy-to-backend connection are separate transport conversations. The proxy injects diagnostic metadata such as X-Request-ID and returns X-Served-By and X-LB-Latency-Ms to support troubleshooting.

Configuration is centralized in config.yaml. The supplied default uses Round Robin, a 1-second connect timeout, a 5-second request timeout, retry_count 1, a 2-second health interval, a 1-second health timeout, an unhealthy threshold of 3, a healthy threshold of 2, and a configured slow_start_s value of 10. The presence of a configuration value should not be interpreted as proof that every related behavior is fully implemented; slow-start ramping requires separate verification.

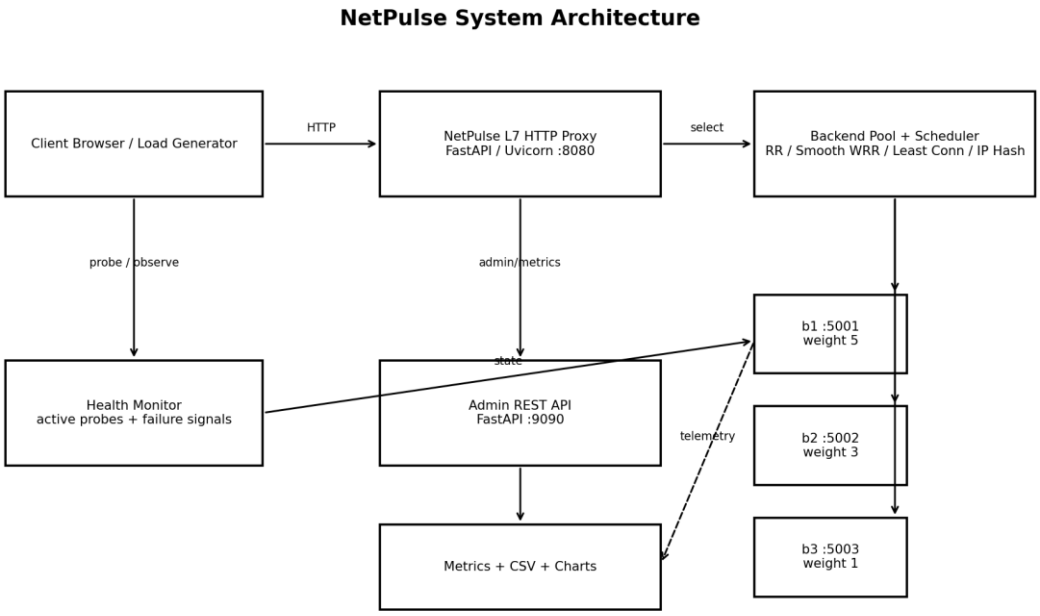


Figure 2.1 NetPulse system architecture.

3 Implementation and Technical Working

3.1 Methodology

The implementation is organized into modules with separate responsibilities. Configuration is loaded first; the backend registry is created from the YAML configuration; scheduling strategies are made available to the proxy; the health monitor begins periodic checks; and the proxy and Admin API are started as ASGI services. Requests enter the L7 proxy, an eligible backend is selected, the upstream response is returned to the client, and request metrics are updated.

Development and verification follow a test-first workflow at the module level: algorithm behavior, health transitions, failure responses, admin operations and Demo Store routes are covered by pytest tests. End-to-end behavior is also exercised with load generators, controlled backend failures and packet capture.

3.2 Core Modules

Module / file	Responsibility
loadbalancer/main.py	Builds and starts the proxy and Admin API applications; serves dashboard and Demo Store static routes.
loadbalancer/proxy.py	Asynchronous HTTP forwarding, backend selection, retries/timeouts, forwarding headers, response metadata and metrics hooks.
loadbalancer/pool.py	Backend registry, backend state, configured/effective weights, active connection counts, add/remove/weight/drain operations.
loadbalancer/config.py	Configuration data structures and YAML loading.
loadbalancer/health.py	Active HTTP health probes, consecutive-success/failure thresholds, state changes and passive failure handling.
loadbalancer/admin.py	REST routes for listing/adding/removing backends, weight changes, drain/undrain, algorithm switching, metrics, health and development failure simulation.
loadbalancer/metrics.py	Request/error/retry counts, latency sample windows, percentiles and backend-level statistics.
loadbalancer/algorithms/	Round Robin, Smooth Weighted Round Robin, weighted Least Connections and IP Hash strategies.
backend/server.py	Configurable upstream HTTP server with health, product/order APIs, artificial delay and failure simulation options.
backend/products.py / backend/orders.py	Shared product catalogue and simulated order validation/persistence.
app/	NetPulse Demo Store interface, cart interactions, search/filtering and live network telemetry UI.
dashboard/index.html	Observability dashboard, backend cards, algorithm controls and Chart.js graphs.
loadbalancer/tcp_proxy.py	Optional asynchronous raw TCP stream proxy.
tools/	Load generation, algorithm comparison, failure simulation, result analysis and optional Mininet topology.
tests/	Algorithm, health, failover, admin integration, Demo Store and proxy error tests.

3.3 Scheduling Algorithms

Round Robin advances through eligible backends in circular order. It is simple and provides approximately equal request counts when the pool remains stable and requests are counted equally.

Smooth Weighted Round Robin accumulates each backend's configured weight into a current score, selects the largest score, and subtracts the total weight from the selected backend. With weights 5:3:1 and all nodes healthy, the expected long-run shares are approximately 55.56%, 33.33% and 11.11%. This is a distribution expectation, not a promise about every short request burst.

Weighted Least Connections chooses a backend based on active connection load normalized by effective weight. It can account for unequal request duration better than pure request-count rotation, but selection and connection acquisition must be interpreted under concurrency; an implementation with separate selection and counter updates can have brief races.

IP Hash uses a deterministic hash of the client address to choose a backend, providing basic session affinity while the pool is unchanged. It is not consistent hashing: adding or removing backends may remap clients.

3.4 Health Monitoring and Fault Tolerance

The health monitor periodically requests each backend's /health endpoint. Consecutive failure thresholds dampen transient failures: a healthy backend can become SUSPECT after a failed check and UNHEALTHY after repeated failures. A backend with sufficient successful probes can return to HEALTHY. Passive proxy errors can also signal a backend failure. The pool's eligibility rules prevent non-healthy or draining backends from receiving normal scheduled traffic.

The repository includes the RECOVERING state in failure-handling logic, but the normal successful recovery path currently transitions directly to HEALTHY. The configuration also includes slow_start_s, but a gradual effective-weight ramp should not be claimed unless it is implemented and measured. The Admin API exposes development failure/restore routes for demonstrations; these are simulations and are distinct from terminating and restarting a real operating-system process.

Graceful draining sets a backend to DRAINING and makes its effective weight zero, so new requests should avoid it while active requests complete. Removal is constrained by active connection state. Runtime testing is required to confirm in-flight request completion and to ensure undrain does not prematurely mark a failed backend healthy.

3.5 Admin API and Observability

The Admin API runs separately on port 9090. Its routes cover backend listing and addition, removal, weight updates, drain and undrain, algorithm selection, metrics retrieval/reset, CSV download, health status and development failure/restore operations. CORS middleware permits configured local development origins so the dashboard served on port 8080 can call the Admin API on port 9090.

The dashboard displays aggregate requests, throughput estimates, error rate, latency summaries, backend health and distribution charts. The browser-side Demo Store uses the proxy origin on port 8080 rather than contacting the backend ports directly. The proxy response headers X-Served-By, X-LB-Latency-Ms and X-Request-ID make each interaction easier to trace.

3.6 Demo Store and Order Processing

The Demo Store is available at /app and presents networking equipment products, search/category filtering, a shopping cart, simulated checkout and a Live Network Activity view. Its API includes GET /api/products, GET /api/products/{product_id}, GET /api/categories, POST /api/orders and GET /api/demo-info. The catalogue is shared through backend/products.py so each backend returns the same product definitions.

Order processing validates customer fields and cart items, looks up products, calculates subtotal/tax/shipping, generates an order identifier and writes the latest orders to results/orders.json. The payment note explicitly states that the transaction is simulated and no real payment is required. The implementation uses a thread lock around file load/save operations, but the complete read-modify-write sequence is not protected by one transaction-wide lock; concurrent order submissions can therefore require further hardening. The current order API should not be described as idempotent without an idempotency-key mechanism and corresponding tests.

3.7 Metrics and Benchmarking

The metrics collector records client requests, failures, retries, status codes, per-backend counts and latency samples. Latency sample deques are bounded to limit memory growth from retaining every sample indefinitely. P50, P95 and P99 percentiles describe latency distribution; P95 is the value at or below which approximately 95% of recorded requests fall.

The benchmark tool runs selected scheduling algorithms under a configured request count and concurrency, writes a CSV and generates a four-panel Matplotlib plot. The saved comparison file contains a 90-request, concurrency-9 run for Round Robin, Smooth Weighted Round Robin and Least Connections. IP Hash is implemented but is not included in that saved comparison. Benchmark numbers are environment-dependent and should be interpreted as one observed run.

3.8 Layer-4 TCP Proxy and Packet Analysis

The optional TCP proxy listens on port 8081 and forwards raw byte streams to configured upstream targets. Unlike the L7 HTTP proxy, it does not parse HTTP methods, paths or headers and therefore cannot apply HTTP header rewriting. It is useful for comparing application-layer routing with transport-layer stream forwarding.

The supplied Wireshark guide describes loopback capture with the Npcap loopback adapter on Windows or the loopback interface on Linux. Useful display filters include tcp.port == 8080, tcp.port in {5001..5003}, tcp.flags.reset == 1, tcp.flags.syn == 1 && tcp.flags.ack == 0, and http.response.code >= 500. Actual packet visibility depends on interface, capture configuration and whether HTTP traffic is dissected.

3.9 Execution Procedure

```
PowerShell — setup and run

python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

```
Terminal 1 — start backend fleet
python tools/failure_sim.py demo

Terminal 2 — start proxy and Admin API
python -m loadbalancer.main

Browser routes
http://127.0.0.1:8080/app
http://127.0.0.1:8080/dashboard

Terminal 3 — run tests and benchmark
python -m pytest -q
python tools/compare_algorithms.py --requests 90 --concurrency 9
```

To test a single request, use curl.exe http://127.0.0.1:8080/test. To inspect administrative data, request http://127.0.0.1:9090/admin/backends or the relevant Admin API endpoint. For the optional L4 proxy, use the command documented in README.md and confirm that its target services are running.

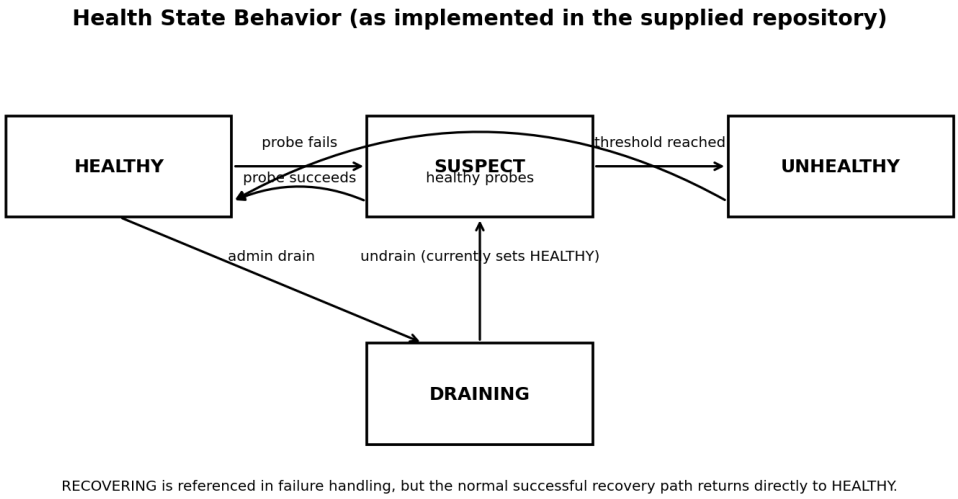


Figure 3.1 Health state behavior as implemented in the supplied repository.

4 Testing and Results

4.1 Test Plan

The test environment used the source files extracted from the supplied CN PROJECT(2).zip archive. The project declares Python 3.10+ and dependency ranges in requirements.txt. The automated test suite was executed in the review environment using python -m pytest -q. The observed result was 44 passed in 7.18 seconds.

Test ID	Test and input	Expected result	Observed result	Status
T1	Run full pytest suite.	All tests pass.	44 passed in 7.18 s in review environment.	Pass
T2	Round Robin algorithm tests.	Eligible backends selected in circular order.	Covered by tests/test_algorithms.py; suite passed.	Pass
T3	Smooth WRR and	Selection follows proportional weights	Covered by algorithm tests; benchmark sample	Pass

	configured weights.	over a sequence.	shows 50/30/10 distribution for 90 requests.	
T4	Least Connections and IP Hash tests.	Selection follows active-load/hash strategy.	Covered by algorithm tests; suite passed.	Pass
T5	Health threshold and failover tests.	Failure changes eligibility; healthy nodes continue serving.	Covered by tests/test_health.py and tests/test_failover.py; suite passed.	Pass
T6	Admin API integration tests.	Backend and algorithm controls return expected responses.	Covered by tests/test_integration.py; suite passed.	Pass
T7	Demo Store route and order tests.	Catalogue, filtering, validation and order persistence are checked.	Covered by tests/test_demo_store.py; suite passed.	Pass
T8	Proxy error tests.	502/503/504 handling, bounded buffers and passive failure signaling.	Covered by tests/test_proxy_errors.py; suite passed.	Pass
T9	Benchmark CSV snapshot.	90 requests per algorithm; record success/error, latency and throughput.	All 90 succeeded in each of three saved runs; see Table 4.2.	Pass for saved run
T10	Browser and physical packet capture.	Dashboard controls and packet flows match documentation.	Not independently executed as part of this report generation.	Pending live verification

4.2 Results and Execution Evidence

Table 4.2 transcribes the saved results/algorithm_comparison.csv in the supplied archive. The workload used 90 requests at concurrency 9 for each algorithm. All three runs recorded 90 successful requests and zero failed requests. The results are a snapshot from the supplied file, not a new benchmark run during report generation.

Algorithm	Success / failed	Avg latency (ms)	P50 (ms)	P95 (ms)	P99 (ms)	Throughput (req/s)	Backend distribution
round_robin	90 / 0	213.71	206.42	390.49	499.82	39.44	{"b1": 30, "b2": 30, "b3": 30}
weighted_rr	90 / 0	265.48	140.09	1231.09	1309.05	32.52	{"b1": 50, "b2": 30, "b3": 10}
least_conn	90 / 0	101.91	81.88	156.81	227.66	78.64	{"b1": 61, "b2": 24, "b3": 5}

4.3 Interpretation of Benchmark Results

In this saved run, Least Connections had the highest measured throughput (78.64 requests/s) and lowest average latency (101.91 ms) among the three tested strategies. Round Robin measured 39.44 requests/s with 213.71 ms average latency. Smooth Weighted Round Robin measured 32.52 requests/s with 265.48 ms average latency and a P95 of 1231.09 ms. The configured backend delays are unequal (50, 100 and 200 ms), and the strategies distribute requests differently; therefore these results should not be generalized beyond the tested environment.

Round Robin distributed the 90 requests evenly (30 per backend). Smooth Weighted Round Robin produced 50, 30 and 10 requests across b1, b2 and b3, exactly matching the configured 5:3:1 ratio for this sample. Least Connections sent 61, 24 and 5 requests respectively, reflecting a dynamic selection policy rather than a fixed proportional schedule. These observations illustrate that a scheduling strategy can affect latency and throughput when backend response times differ.

4.4 Objective Achievement

Objective	Evidence	Outcome
O1	Algorithm unit tests and saved distribution results.	Achieved for implemented algorithms.
O2	Health/failover unit tests and passive failure tests.	Partially demonstrated by automated tests; repeat live process-failure test before claiming zero dropped requests.
O3	Admin integration tests for backend and algorithm controls.	Achieved in test suite.
O4	Metrics tests, saved benchmark CSV and plot.	Achieved for recorded metrics and three-algorithm comparison; IP Hash is not in saved benchmark.
O5	Demo Store tests and proxy route implementation.	Achieved at code/test level; verify browser UI live before submission.
O6	TCP proxy module and Wireshark guide.	Implemented as an extension; live packet capture remains to be attached if required.

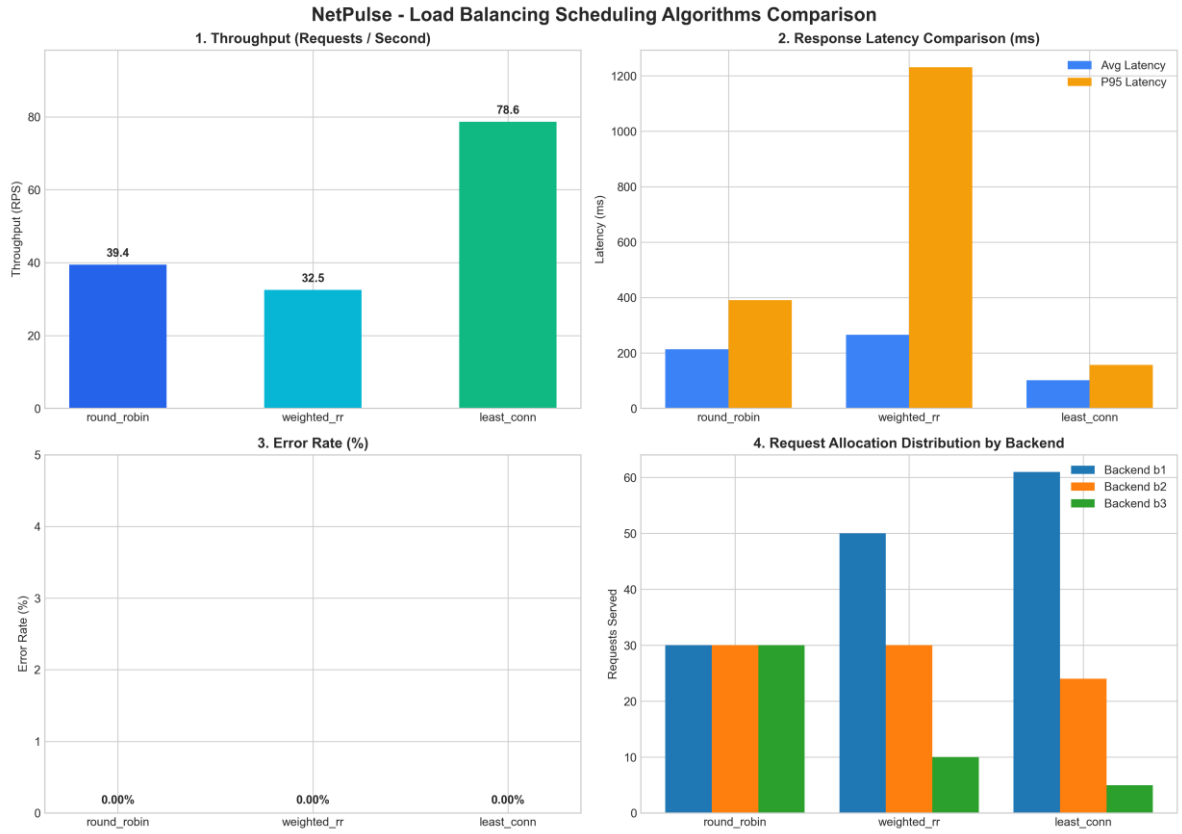


Figure 4.1 Saved benchmark plot from results/algorithm_comparison.png.

5 Discussion and Conclusion

5.1 Discussion and Comparative Analysis

The project demonstrates that load balancing is not only a choice of algorithm; it also depends on backend capacity, request duration, health state and workload concurrency. Round Robin is predictable and easy to explain. Smooth Weighted Round Robin allocates more requests to higher-weight nodes while interleaving selections. Least Connections can improve distribution when requests occupy connections for different durations, although results depend on how active connections are tracked and on the workload. IP Hash provides basic stickiness but can remap clients when the backend pool changes.

The saved benchmark indicates that Least Connections performed best on throughput and average latency for this particular local run. Smooth Weighted Round Robin's tail latency was much higher in this sample. No algorithm is universally best: the conclusion depends on backend delays, workload, concurrency, warm-up, connection pooling and measurement methodology. Repeated runs with consistent settings should be performed for stronger comparisons.

5.2 Advantages and Applications

NetPulse provides a single client endpoint, multiple scheduling strategies, health-based backend exclusion, runtime administrative controls, request tracing headers, latency metrics and a visual monitoring surface. The Demo Store provides realistic HTTP traffic beyond a synthetic /test endpoint.

The separate TCP proxy makes the L4/L7 distinction tangible, while the Wireshark workflow links application behavior to TCP handshakes, resets and HTTP exchanges.

Related use cases include internal service gateways, development test environments, API front doors, backend pool experiments and teaching demonstrations of reverse proxy behavior. The current implementation is a lab prototype, not a production-ready gateway.

5.3 Limitations

The project runs a single load-balancer process, so that process is a single point of failure. Backend registry and metrics are primarily in memory and are lost on restart unless separately persisted. The L4 proxy uses its own target configuration and does not provide equivalent health-aware failover. The current saved benchmark covers three algorithms, not IP Hash, and is based on one run.

The normal health recovery path returns directly to HEALTHY; a complete RECOVERING/slow-start weight ramp needs additional implementation and evidence. The order store uses JSON-file persistence and thread locks, but the read-modify-write transaction and duplicate-submission semantics need hardening for concurrent use. The project does not provide real payment processing, TLS termination, authentication for the Admin API, distributed coordination, or production-grade multi-process storage.

5.4 Challenges and Troubleshooting

Issue	Diagnosis	Solution / verification
Dashboard CORS errors	Dashboard origin :8080 and Admin API origin :9090 use different ports.	Configure local CORS origins on the Admin API; verify browser preflight and actual requests.
Backend not receiving requests	Backend process stopped, health state ineligible, or incorrect host/port.	Check config.yaml, each /health endpoint and Admin API backend state.
Unexpected latency percentiles	Backend delays differ and concurrent tasks contend for the slower nodes.	Use the same request count/concurrency, warm-up and repeat runs; interpret P95/P99 separately from averages.
Failure simulation differs from real crash	Admin development endpoints modify simulated state rather than terminating an OS process.	Use tools/failure_sim.py or stop the backend process to test actual connection failure.
Order duplication risk	POST can be resubmitted; no idempotency-key contract is evidenced in order module.	Add a client idempotency key and atomic order transaction; test duplicate submissions.
Wireshark shows no local traffic	Capture interface may not be the loopback adapter or HTTP dissection may be unavailable.	Select Npcap loopback on Windows or lo on Linux and use TCP port filters first.

5.5 Future Scope

Future improvements include a fully implemented RECOVERING state with gradual slow-start weight ramping; health-aware L4 target selection; safe idempotency-key handling for orders; transactional or database-backed order storage; authentication and authorization for administrative actions; TLS termination; consistent hashing; dynamic configuration/service discovery; more robust header sanitization; structured logs and distributed tracing; repeated benchmark runs with confidence

intervals; IP Hash in the comparison suite; Mininet delay/loss experiments; and deployment of multiple proxy instances behind a highly available front end.

5.6 Conclusion

NetPulse implements a modular academic demonstration of an asynchronous HTTP reverse proxy and load balancer with four scheduling strategies, health monitoring, an Admin REST API, metrics, a dashboard, a Demo Store, a TCP stream proxy and automated tests. The supplied source archive passed 44 pytest tests in the review environment. Its saved 90-request comparison showed zero failures for the three benchmarked algorithms and measurable differences in throughput, latency and backend distribution. The results support the value of observing both scheduling behavior and latency distribution rather than relying on request counts alone. Remaining work includes stronger real-process failure evidence, complete recovery/slow-start behavior, transaction-safe order handling, and repeated live validation before production-like reliability claims are made.

References

[1] R. Fielding, M. Nottingham, and J. Reschke, “HTTP Semantics,” RFC 9110, Internet Engineering Task Force, June 2022. https://www.rfc-editor.org/rfc/rfc9110

[2] Python Software Foundation, “asyncio — Asynchronous I/O.” https://docs.python.org/3/library/asyncio.html

[3] Encode, “HTTPX Documentation.” https://www.python-httpx.org/

[4] FastAPI, “FastAPI Documentation.” https://fastapi.tiangolo.com/

[5] pytest development team, “pytest Documentation.” https://docs.pytest.org/

[6] Wireshark Foundation, “Wireshark User’s Guide.” https://www.wireshark.org/docs/wsug_html_chunked/

[7] MIT World Peace University, Department of Computer Engineering and Technology, “Final Computer Networks Mini Project Report Template,” course CSE20110, academic year 2026–2027.

[8] NetPulse project source archive supplied for this report: CN PROJECT(2).zip, including README.md, config.yaml, loadbalancer/, backend/, app/, dashboard/, tools/, tests/, results/ and docs/.

AI Usage Declaration

We used ChatGPT to help organize and draft report prose from the supplied project template and source archive, summarize the code architecture, and format the report. The generated content was checked against the supplied files; the pytest suite was executed in the review environment and the saved benchmark CSV was transcribed. The team must verify all names, PRNs, faculty/group details, run the project in its own environment, and attach any required live screenshots or packet captures

before submission. Responsibility for the final report and submitted implementation remains with the students.

Appendices

Appendix A — Key Configuration

```
server: 127.0.0.1:8080
admin: 127.0.0.1:9090
algorithm: round_robin
connect_timeout_s: 1.0
request_timeout_s: 5.0
retry_count: 1
health interval: 2.0 s
health timeout: 1.0 s
unhealthy threshold: 3
healthy threshold: 2
slow_start_s: 10
b1: 127.0.0.1:5001, weight 5, delay 50 ms
b2: 127.0.0.1:5002, weight 3, delay 100 ms
b3: 127.0.0.1:5003, weight 1, delay 200 ms
```

Appendix B — Principal Project Structure

```
CN PROJECT/
├── config.yaml, requirements.txt, README.md
├── loadbalancer/
│   ├── main.py, proxy.py, pool.py, config.py
│   ├── health.py, metrics.py, admin.py, tcp_proxy.py
│   └── algorithms/{round_robin,weighted_rr,least_conn,ip_hash}.py
├── backend/{server,products,orders}.py
├── app/{index.html,assets/app.js,assets/styles.css}
├── dashboard/index.html
├──
tools/{load_gen,compare_algorithms,failure_sim,failure_test,analyze_results,runtime_demo
,mininet_topo}.py
├──
tests/{test_algorithms,test_demo_store,test_failover,test_health,test_integration,test_p
roxy_errors}.py
├──
results/{algorithm_comparison.csv,algorithm_comparison.png,load_test.csv,orders.json}
└── docs/{TEAM_VIVA_GUIDE.md,wireshark_guide.md}
```

Appendix C — Team Contributions

The table below is a suggested module ownership map from the project guide. Replace names and evidence with the work each student actually completed; do not submit the role allocation as a claim unless the team confirms it.

Student / PRN	Suggested module ownership	Evidence / deliverables to confirm
Naitik Sharma	Load balancing engine and scheduling	proxy.py, algorithms/,

	algorithms	tests/test_algorithms.py
Purva Shetiya	Health monitoring and fault tolerance	health.py, pool.py, tests/test_health.py, tests/test_failover.py
Tanushree Walunj	Demo Store and observability dashboard	app/, backend/products.py, backend/orders.py, dashboard/index.html, tests/test_demo_store.py
Shubham Ojha	Metrics, benchmarking and network analysis	metrics.py, tools/compare_algorithms.py, tools/load_gen.py, tcp_proxy.py, docs/wireshark_guide.md

Appendix D — Submission Checklist

Before submitting, fill in all four student names and PRNs, group number, panel, batch, faculty name and actual submission date. Verify the report against the final local repository, update the table of contents in Word, insert required screenshots of the running Demo Store/dashboard and test output, attach Wireshark captures if required, and ensure no instructional template placeholders remain.

Student Declaration and Signatures

We take responsibility for the accuracy of this report and understand the submitted implementation and results. Sources and assistance used have been acknowledged.

Student Name	PRN	Signature
Naitik Sharma	1262243087	_____
Purva Shetiya	1262243040	_____
Tanushree Walunj	1262243564	_____
Shubham Ojha	1262241817	_____